

1 Lesson 29-09-26

Branches: we have to quickly finish up on branching instructions in the RISC-V ISA, which we left behind on the past lesson.

These are a special class of instructions: range of ± 4 KiB and a 12-bit immediate value.

beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch $\geq$	B	1100011	0x5		if(rs1 $\geq$ rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch $\geq$ (U)	B	1100011	0x7		if(rs1 $\geq$ rs2) PC += imm	zero-extends

The offset must be a multiple of 2, and it is added to PC on positive comparisons. For this, branch instructions compare two registers Rs1 and Rs2

The types are a bit limited:

- **beq** and **bne** take the branch if *rs1* and *rs2* are equal or unequal;
- **blt** and **bltu** take the branch if *rs1* is less than *rs2*;
- **bge** and **bgeu** take the branch if *rs1* is greater than or equal to *rs2*;
- **bgt**, **bgtu**, **ble**, and **bleu** can be synthesized by reversing the operands to **blt**, **bltu**, **bge**, and **bgeu**. The compiler does this automatically.

Some zero-based pseudo-instructions are available:

- **beqz** *rs*, *label*: branches if *rs* = 0:

```
1 beqz x3, label
2
3 # translates to:
4
5 beq x3, x0, label
```

- **bnez** *rs*, *label*: branches if *rs* != 0:

```
1 bnez x3, label
2
3 # translates to:
4
5 bne x3, x0, label
```

- **bltz** *rs*, *label*: branches if *rs* < 0:

```
1 bltz x3, label
2
3 # translates to:
4
5 blt x3, x0, label
```

- **bgtz** *rs*, *label*: branches if *rs1* $\geq$ 0:

```
1 bgtz x3, label
2
3 # translates to:
4
5 bgt x3, x0, label
```

1.1 Pipelining

By studying RISC-V, we figured out that the CPU reads a sequence of fixed size instructions from memory, executing them one by one. We now introduce an implementation technique whereby multiple instructions are overlapped in execution: we call this technique **pipelining**. In a pipeline, different units (called pipe *stages* or *segments*) are (at any given time) completing different parts of different instructions in parallel.

Different stages of the pipeline perform different fixed functions on instructions. For instance, we might have (in an undefined implementation) that the execution of each instruction may be composed of at most five clock cycles:

- Instruction fetch cycle (IF)
- Instruction decode/register fetch cycle (ID)
- Execution/effective address cycle (EX)
- Memory access/branch completion cycle (MEM)
- Write-back cycle (WB).

1.1.1 Pipeline metrics

Let's define some *metrics*.

The **throughput** of a pipelined processor is the number of instructions which exit the pipeline in the time unit.

All the pipeline stages are synchronized (they proceed to executing a new task all together): the time for executing one step is called **machine cycle**, and normally corresponds to one *clock cycle*. The length of the machine cycle is determined by the slowest stage of the pipeline.

Lastly, we define **CPI** as *Clock Cycles per Instruction*.

In an ideal pipeline, all the stages are perfectly balanced (i.e., they require the same time). The throughput of an ideal pipeline (i.e., the number of instructions completed in a given time period) is:

$$\text{throughput}_{\text{pipelined}} = \text{throughput}_{\text{unpipelined}} * n$$

where n is the number of stages.

1.1.2 Pipeline performance

Pipelining increases the processor *throughput* without making single instructions execution faster. In fact, single instruction processing is made slower due to the pipeline control *overhead*. The depth of a pipeline is limited by:

- The need for balanced stages;
- Pipelining overhead (pipeline register delay and clock skew).

1.2 Pipeline hazards

Hazards are situations that prevent an instruction from executing during its designated clock cycle. There are three classes of hazards:

- **Structural hazards:** coming from resource conflict;
- **Data hazards:** an instruction depends on the result of a previous instruction;
- **Control hazards:** depend on pipelining branches and other instructions that change the PC.

Let's see ways to deal with these hazards.

1.2.1 Stalling

One way of dealing with hazards is to force the pipeline to **stall**, i.e., to block instructions for one or more clock cycles.

When an instruction is stalled:

- The instructions following the stalled instruction are also stalled;
- The instructions preceding the stalled instruction continue.

This can be easily achieved by modifying the buffer registers, making them able to *conserve* the previous value (this will be the stage which caused the hazard), and introducing an empty instruction to propagate onwards.

A stall causes the introduction of a "*bubble*" in the pipeline, which has to propagate through the whole pipeline.

This is the simplest way to solve pipeline hazards, as waiting it out (eventually) solves all problems.

1.2.2 Structural hazards

Structural hazards may happen when some pipeline unit is not able to execute all the operations scheduled for a given cycle. This is usually tied to limited resources in the processor.

Some common examples are:

- A given unit is not able to complete its task in one clock cycle;
- The pipeline owns only one register-file write port, but there are cycles in which two register writes are required;
- The pipeline refers to a single-port memory, and there are cycles in which different instructions would like to access to the memory together.

Solution

Removing structural hazards requires adding new hardware or improving the existing one. The designer has to trade-off between performance (add more resources or keep stalling) and cost (add more resources or keep the ones you have), basing on the frequency of occurrence of structural hazards.

This calls for optimization. Let's consider *load structural hazards*: these happen when two instructions contemporarily try to make a memory access to a single-port memory. The clear solution is to split instruction and data memory.

Assume that:

- 40% of instructions access the memory;
- The machine with the structural hazard has a clock 1.05 times faster than the one without.

We want to measure the performance gain.

- For the machine without structural hazard:

$$\text{Average Instruction Time}' = CPI \times T_C$$

where T_C is the clock period.

- For the machine with structural hazard:

$$\text{Average Instruction Time}'' = CPI_{\text{ideal}} \times T_c'' + \text{Inst. freq} \times \text{Stall penalty}$$

computed using $T_c'' = T_C \times 1.05$.

1.2.3 Data hazards

Overlapping the execution of instructions, as it is done by pipelining, changes the order of read/write accesses to operands: This can result in:

- Wrong results;
- Undeterministic behavior.

We call these **data hazards**.

Let's consider the code fragment:

```

1 add r1, r2, r3 # r1 write
2
3 sub r4, r1, r5 # r1 read from here onwards !
4 and r6, r1, r7
5 or  r8, r1, r9
6 xor r10, r1, r11

```

The problem is that the ADD instruction writes the result in R1 in the WB stage.

- The SUB instruction reads R1 in its ID stage, two clock cycles before the one in which R1 is made available (one to feed R1 to WB and one to let WB finish): this produces a wrong result;
- The AND instruction reads R1 in its ID stage, one clock cycle before the one in which R1 is made available: this produces a wrong result;
- The OR instruction reads R1 in its ID stage, in the same clock cycle in which R1 is made available: this produces a correct result, provided that the write accesses to the register file are made before the read accesses. From now onwards, until we write R1 again, the results are correct (XOR is correct).

Another scenario in which we encounter data hazards is because of *interrupt effects*. If an interrupt occurs during the execution of a critical piece of code (from the point of view of data hazards), correctness may actually be restored (as instructions leave the pipeline as the interrupt is handled). This may cause an undeterministic behavior.

Solution

The typical solution to data hazards is that of **forwarding**. A special hardware in the datapath detects when a previous ALU operation should write the register corresponding to the source of the current ALU operation. In this case, the hardware selects the ALU result as the ALU input rather than the value from the register file.

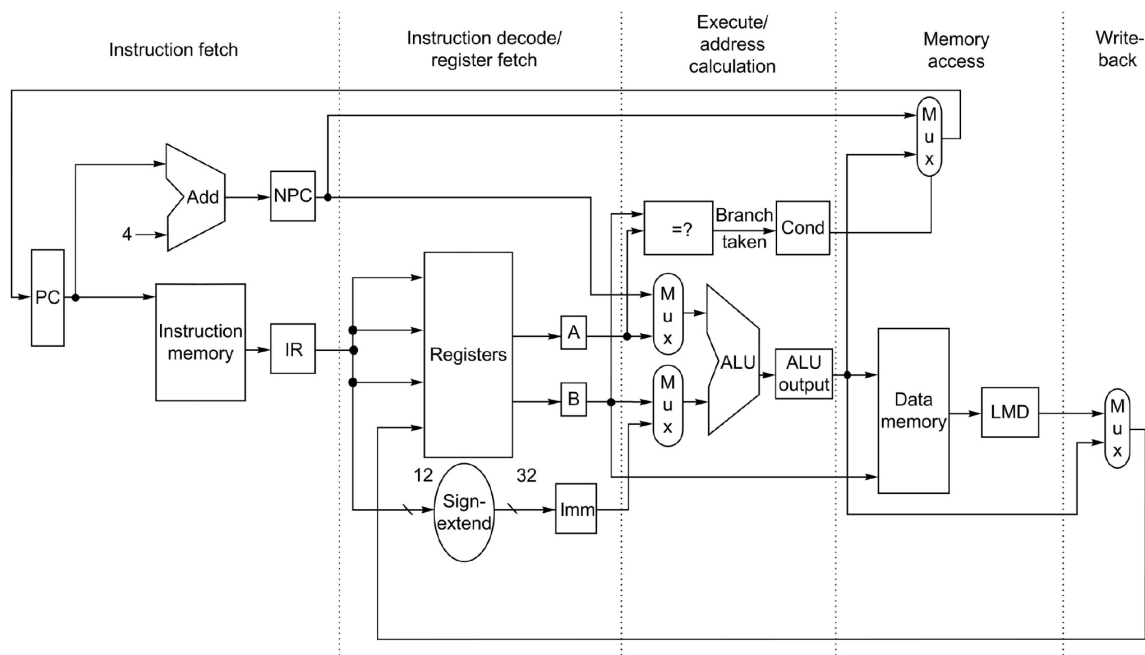
For this to be implemented, the hardware must be able:

- To forward a data from any of the previously started instructions (provided that they didn't already write the data in its final location);
- Not to forward anything, if the following instruction is stalled, or an interrupt occurred.

Forwarding can also be **generalized**: in order to always avoid stalling, it should be made possible between any pipeline register to any input of any functional unit.

1.3 Datapath model

Let's look at a simplified **datapath** schematic to explain pipelines with more detail.



1.3.1 Instruction fetch

The **instruction fetch** stage has the job of fetching the next instruction from program memory and feeding it to the next stage.

As we can see, we use the **PC** (*Program Counter*) to index instruction memory and put the instruction data into the **IR** (*Instruction Register*). Then we increment the program counter by 4 bytes (the size of a RISC-V instruction) and put that into an **NPC** (*New Program Counter*) register.

1.3.2 Instruction decode

We then have to **decode** the instruction.

A number of fields are extracted from the instruction encoding (we saw that RISC-V is highly regular). This allows for *fixed-field decoding*: decoding can be performed while registers are read.

- Parts of the instruction that index *registers* index the **register file**. In this example we index two registers, *A* and *B*;
- The immediate operand is also extracted from the instruction encoding, sign extended and sent to the next stage.

We can say that this stage performs both **decoding** and **register fetching**.

1.3.3 Instruction execution

At this point we can **execute** the instructions.

This is performed through the **ALU** (*Arithmetic Logic Unit*), which is basically a big multiplexer of different computation circuits.

We also have to distinguish between input registers and immediates (through multiplexers). Let's look at a couple of instruction types as an example:

- *Memory reference*: $\text{ALU output} \leftarrow A + \text{Imm}$. Note that we store the target *address* of the memory access! For loads we will use this directly, for stores we will use *B* and write the ALU output;
- *Register-Register ALU instruction*: $\text{ALU output} \leftarrow A \text{ funct } B$
- *Register-Register ALU instruction*: $\text{ALU output} \leftarrow A \text{ funct Immediate}$
- *Branch*: $\text{ALU output} \leftarrow \text{NPC} + \text{Immediate} \ll 2, \text{Cond} \leftarrow (A == B)$

Another thing to note is if the instruction is a branch one: if it is, we have to evaluate a condition and use it to multiplex between NPC and a calculated value (here the value of register *A*).

We can say that this stage performs both **execution** and **address calculation**.

1.3.4 Memory access

An extra stage is given by the **memory access** stage.

- If a (**load**) instruction requires it, we use the ALU output to address memory and load the needed contents in the **LMD** (*Loaded Memory Data*) register;
- If a (**store**) instruction requires it, we use the *B* register to address memory, and write the ALU output to it.

This is also the stage where we perform the *branch completion cycle*, that is decide if we should update the NPC or keep it at $\text{PC} + 4$.

More clearly:

- *Memory reference*: $\text{LMD} \leftarrow \text{Mem}[\text{ALUOutput}]$ (loads) or $\text{Mem}[\text{ALUOutput}] \leftarrow B$ (stores);
- *Branch*:

$$\text{cond} ? \text{PC} \leftarrow \text{ALU output}, \text{ otherwise } \text{PC} \leftarrow \text{NPC}$$

1.3.5 Write-back cycle

The last step to perform is the **write-back**.

Based on the instruction type, we update the register file with the obtained result:

- *Register-to-register* or *register-to-memory* instructions feed the ALU output to the register file;
- *Load* instructions feed the LMD to the register file.

More clearly:

- *Register-Register ALU instruction*: $\text{Regs}[IR11.7] \leftarrow \text{ALU output}$;
- *Register-Immediate ALU instruction*: $\text{Regs}[IR11.7] \leftarrow \text{ALU output}$;
- *Load instruction*: $\text{Regs}[IR11.7] \leftarrow \text{LMD}$;

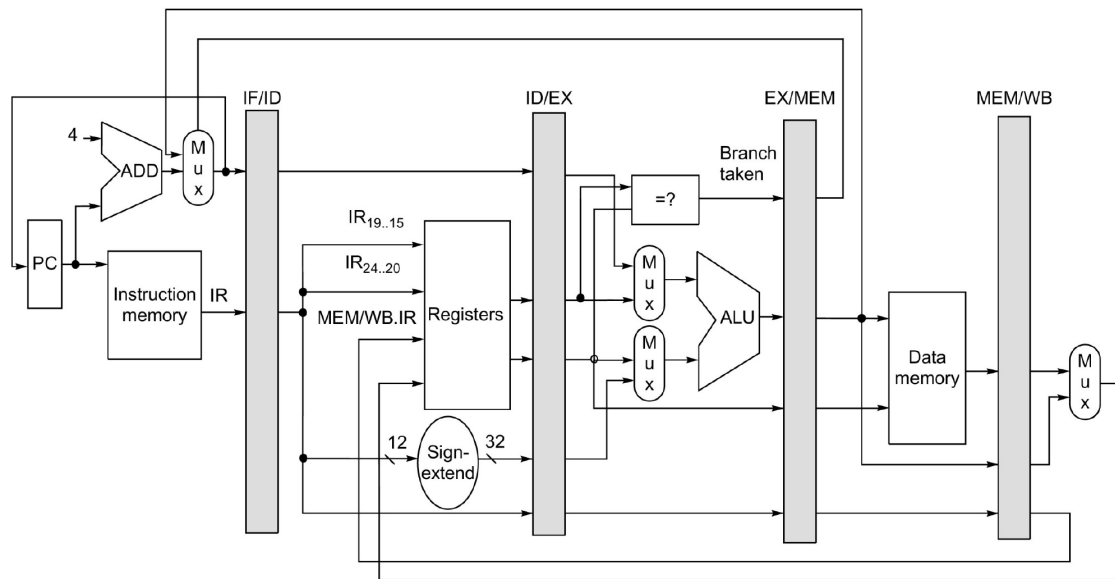
In this discussion we skipped a lot of details about the implementation, for instance how the multiplexers are driven, etc... We assume these details to be (more or less) easy to synthesize as combinatorial or sequential networks once an actual architecture is defined.

In the described architecture, all instructions require 5 clock cycles, except branch instructions, which require 4 clock cycles. Optimizations could be done for reducing the average CPI: as an example, the ALU instructions could be completed during the memory access cycle. Also, hardware resources could be optimized by avoiding duplications (e.g., for ALUs, and memory). An alternative single-clock architecture (i.e., executing an instruction per clock cycle) can also be considered, but in that case a simple control unit is required to produce the signals required by the datapath.

1.3.6 Pipelined version

A different solution can be using a **pipelined** version, where a new instruction is started at each clock cycle. Different resources work on different instructions at the same time, and at every clock cycle, each resource can be used for one purpose, only. This means that:

- Separate instruction and data memories (i.e., caches) must be used;
- The register file is used in two stages: for reading (second half of the cycle) in ID and for writing (first half of the cycle) in WB. It must be designed to satisfy these requests during the same clock cycle.
- The PC must be changed in the IF stage, and branch handling must be considered;
- *Pipeline registers* must be added between stages to act as buffers.



1.3.7 Structural hazard control

We don't go in depth on **structural hazard** control as we saw that it depends on the resources available to the processor. The solution remains *stalling*.

1.3.8 Data hazard load control

Let's first go over the **causes** of data hazards.

Causes

A hazard is created whenever there is dependence between instructions, and they are close enough that the overlap caused by pipelining would change the order of access to an operand. In general, this can happen for:

- **Register operands;**
- **Memory operands:** this is possible if:
 - Accesses to memory by load and store are not made in the same stage;
 - Execution can proceed while an instruction waits for a cache miss to be solved.

A problem is that not all data hazards can be solved through data forwarding. Focusing on **loads**, let's consider the code fragment:

```

1 ld r1, 0(r2) # r1 loaded
2
3 sub r4, r1, r5 # r1 used
4 and r6, r1, r7
5 or r8, r1, r9

```

Here, when **sub** enters the execution stage, the data memory access is just being initiated on instruction **ld**. The forwarding path here goes back in time, therefore it is impossible to implement. This means it's impossible to forward: our only solution is a stall.

Load control

Implementing the **control** for these bubbles requires that at each clock cycle:

- All tests for detecting a possible data hazard concerning an instruction are performed when this is in the ID stage
- If a data hazard is detected, two actions can be alternatively taken:
 - The appropriate forwarding is activated;
 - The instruction is stalled before entering the stage where operands are not available (i.e., before being issued).

The control circuitry used to handle loads is called **LID** (*Load Interlock Detection*).

Let's look at a table of possible hazard conditions:

Situation	Example fragment	Action
No dependence	LD R1,45(R2) DADD R5,R6,R7 DSUB R8,R6,R7 OR R9,R6,R7	No hazard possible because no dependence exists on R1 in the immediately following three instructions.
Dependence requiring stall	LD R1,45(R2) DADD R5,R1,R7 DSUB R8,R6,R7 OR R9,R6,R7	Comparators detect the use of R1 in the DADD and stall the DADD (and DSUB and OR) before the DADD begins EX.
Dependence overcome by forwarding	LD R1,45(R2) DADD R5,R6,R7 DSUB R8,R1,R7 OR R9,R6,R7	Comparators detect use of R1 in DSUB and forward result of load to ALU in time for DSUB to begin EX.
Dependence with accesses in order	LD R1,45(R2) DADD R5,R6,R7 DSUB R8,R6,R7 OR R9,R1,R7	No action required because the read of R1 by OR occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half.

We deduce that the following checks have to be performed when a load instruction is in the EX stage and another instruction exploiting the loaded value is in the ID stage.

ID Opcode	IF Opcode	Match
Load	Register-Register ALU	ID/EX.IR[rd] == IF/ID.IR[rs1]
Load	Register-Register ALU	ID/ EX.IR[rd] == IF/ID.IR[rs2]
Load	Load, store, ALU immediate, branch	ID/EX.IR[rd] == IF/ID.IR[rs1]

If operands match, a data hazard is detected and as a result, the control unit must insert a pipeline stall and prevent the instructions in IF and ID stages from advancing.

At this point, given an instruction currently in the ID stage, introducing a stall in the EX stage can be done by:

- Forcing a nop instruction in the ID/EX pipeline register (in RISC-V the nop corresponds to an `addi x0, x0, 0`);
- Forcing the IF/ID pipeline register to maintain the current value (*conserve*);
- Freezing the PC (maintain unaltered IF status).

1.3.9 Data hazard forwarding control

Note that the above table only pertains to *load* operations, that is situations when we are *loading* some data from memory and an instruction wants to access that data while the load is still happening in the pipeline.

We said that there can also be *register* hazards, so now let's shift our attention to a method of solving them, that is **forwarding**. This being said, in a special case (row 4 of the table at 4.3.8) load instructions can also benefit from forwarding from MEM to EX.

SO to summarize, forwarding can be implemented:

- From the ALU or data memory output;
- To ALU inputs, data memory inputs, or the zero detection unit (branch instructions).

The forwarding logic must compare:

- The destination fields of the IR contained in the EX/MEM and MEM/WB registers;
- The source fields of the IR contained in the IF/ID, ID/EX and EX/MEM registers.

We see one final table regarding the forwarding conditions and the *comparisons* (checks):

Source	Opcode	Dest.	Opcode	Forward	Comparison
EX/MEM	Reg-reg ALU, ALU imm.	ID/EX	Reg-reg ALU, ALU imm., load, store, branch	Top ALU	EX/MEM.rd == ID/EX.rs1
EX/MEM	Reg-reg ALU, ALU imm.	ID/EX	Reg-reg ALU	Bottom ALU	EX/MEM.rd == ID/EX.rs2
MEM/WB	Reg-reg ALU, ALU imm., load	ID/EX	Reg-reg ALU, ALU imm., load, store, branch	Top ALU	MEM/WB.rd == ID/EX.rs1
MEM/WB	Reg-reg ALU, ALU imm., load	ID/EX	Reg-reg ALU	Bottom ALU	MEM/WB.rd == ID/EX.rs2

To summarize, we note that the following situations might happen in the pipeline, and the tools we have available to solve them:

Situation	Solution
No dependence	Do nothing
Structural hazard	Stall
Any source IF/ID register in load target ID/EX stage	Stall (table 1)
Any reg-reg ID/EX register in any target EX/MEM stage	Forward (table 2)
Any reg-reg or load ID/EX register in any target MEM/WB stage	Forward (table 2)

Exact register comparisons are found in the tables above. Table 1 refers to the *load control* table, whereas table 2 refers to the *forwarding control* table. It's clear that we are

forced to stall only on structural and load dependencies (so when the data physically isn't in the CPU at the needed time).